PM

Pub Mail

Rodar o Pub Mail localmente

Node na máquina ou tudo no Docker — guia testado do zero

Atualizado em 08/10/2026

Este guia coloca o sistema inteiro para rodar localmente, do zero. São dois jeitos:

- **Jeito A — Node na sua máquina.** O MySQL e o Redis rodam no Docker, e você roda o back, os micros e o front com Node. É o jeito para desenvolver, porque o código recarrega sozinho quando você salva.
- **Jeito B — tudo no Docker.** Um comando sobe banco, Redis, back, micros e front. É o jeito para só ver o sistema funcionando, ou testar uma branch, sem instalar Node.

Se você só quer abrir o sistema, comece pelo Jeito B. Se vai mexer no código, use o Jeito A.

---

1. Versões

Ferramenta	Versão	Observação
Node.js	**22 LTS** (produção usa 22.23.3)	Node 20 não funciona: o Prisma 7 exige Node 22 ou mais novo.
npm	10.9.x (vem junto com o Node 22)	Não é usado para instalar o projeto.
Yarn	**4.10.3**	Vem pelo Corepack, que já faz parte do Node. Cada `package.json` fixa essa versão.
Docker	Docker Desktop recente (Compose v2)	Reserve pelo menos 4 GB de memória para ele.
MySQL / Redis	8.0 / 7	Já vêm no `docker-compose.yml`.

> **O projeto usa Yarn 4, não npm.** Não rode `npm install`. Os arquivos `package-lock.json` que existem em algumas pastas são antigos e instalam versões diferentes das de produção. O que vale é o `yarn.lock`.

Na raiz do repositório há um `.nvmrc` com a versão do Node. Quem usa nvm faz:

```
nvm install      # instala o Node 22 lido do .nvmrc
nvm use
node -v          # v22.x
```

Depois, uma vez só por máquina:

```
corepack enable
yarn -v          # dentro de qualquer pasta do projeto deve mostrar 4.10.3
```

Se `yarn -v` mostrar `1.22.x`, o Corepack não está ativo. No Windows, rode `corepack enable` num terminal aberto como administrador.

---

2. Serviços e portas

Serviço	Pasta	Porta	O que faz
Front (React + Vite)	`front/vite`	**3000**	O painel. Abra http://localhost:3000
Back (NestJS)	`back`	**8000**	A API. Documentação em http://localhost:8000/docs
Micro de IA	`ai-micro-services`	4000	Respostas do webchat com IA. Só o back fala com ele.
Micro de e-mail	`email-micro-services`	9999	Fila de envio de e-mails. Só o back fala com ele.
MySQL 8	Docker	3306	Banco `pubmail`, usuário `pubmail`, senha `pubmail`.
Redis 7	Docker	6379	Filas e cache.
phpMyAdmin	Docker (opcional)	8081	Só sobe com `docker compose --profile tools up -d`. Servidor `mysql`, usuário `pubmail`, senha `pubmail`.

O navegador só conversa com o back (porta 8000). O back conversa com os micros, o MySQL e o Redis.

---

3. Jeito B — tudo no Docker

Pré-requisito: só o Docker Desktop aberto. Não precisa de Node.

```
git clone <url-do-repositório> pub-mail
cd pub-mail
docker compose --profile app up -d --build
```

A primeira vez leva de 5 a 15 minutos, porque baixa as imagens e instala as dependências (num Mac com 4 GB reservados ao Docker levou cerca de 15). Acompanhe o back subindo:

```
docker compose logs -f back
```

Quando aparecer `Nest application successfully started`, abra **http://localhost:3000** e entre com:

```
E-mail: admin@pubmail.local
Senha:  admin12345
```

Esse usuário é criado sozinho na primeira subida, com perfil SUPER_ADMIN, e enxerga todos os módulos. As migrations do banco também rodam sozinhas a cada subida do back.

Comandos do dia a dia

```
docker compose --profile app ps                # o que está rodando
docker compose --profile app logs -f back      # logs de um serviço (back, ai, email, front)
docker compose --profile app up -d --build     # depois de puxar código novo: reconstrói e
sobe
docker compose --profile app down              # para tudo (os dados do banco ficam)
docker compose --profile app down -v           # para tudo e APAGA banco, Redis e uploads
```

No Jeito B o código fica copiado dentro das imagens. Mudou o código? Rode o `up -d --build` de novo.

Trocar senhas, portas ou o usuário inicial

Crie um arquivo `.env` na **raiz** do repositório (ele não vai para o git) com o que quiser trocar:

```
SEED_ADMIN_EMAIL=voce@empresa.com
SEED_ADMIN_PASSWORD=uma-senha-longa
MYSQL_PORT=3307          # se a 3306 já estiver ocupada na sua máquina
JWT_ACCESS_SECRET=qualquer-texto-longo
JWT_REFRESH_SECRET=outro-texto-longo
```

O usuário inicial só é criado se ainda não existir. Para recriar tudo do zero com os valores novos, rode `docker compose --profile app down -v` antes.

---

4. Jeito A — Node na sua máquina

Pré-requisitos: Node 22, Corepack ativo (seção 1) e Docker Desktop aberto. Use um terminal para cada serviço.

4.1 Banco e Redis

Na raiz do repositório:

```
docker compose up -d
docker compose ps        # mysql e redis devem ficar "healthy"
```

Esse comando sobe só o MySQL e o Redis. O banco já é criado com o charset e o fuso que o sistema espera.

4.2 Back

```
cd back
yarn install
```

Crie o arquivo `back/.env` com este conteúdo:

```
PORT=8000
DATABASE_HOST=127.0.0.1
DATABASE_PORT=3306
DATABASE_USER=pubmail
DATABASE_PASS=pubmail
DATABASE_NAME=pubmail
DATABASE_URL=mysql://pubmail:pubmail@127.0.0.1:3306/pubmail
DATABASE_POOL_LIMIT=10
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
JWT_ACCESS_SECRET=dev-local-access-secret
JWT_REFRESH_SECRET=dev-local-refresh-secret
JWT_ACCESS_EXPIRES_IN=15m
JWT_REFRESH_EXPIRES_IN=7d
IA_SERVICE_URL=http://127.0.0.1:4000
IA_SERVICE_KEY=dev-ia-key
EMAIL_SERVICE_URL=http://127.0.0.1:9999
EMAIL_SERVICE_KEY=dev-email-key
PUBLIC_API_URL=http://localhost:8000
BASE_API_URL=http://localhost:8000
WEBCHAT_BACKEND_INTERNAL_URL=http://127.0.0.1:8000
WEBCHAT_DOMAIN_BIND_PROVIDER=NONE
STRIPE_SECRET_KEY=sk_test_disabled_placeholder
SWAGGER_ENABLED=true
```

Prepare o banco e crie seu usuário:

```
yarn prisma migrate deploy      # cria as tabelas
yarn prisma generate            # gera o client do Prisma em back/generated

SEED_ADMIN_EMAIL=admin@pubmail.local SEED_ADMIN_PASSWORD=admin12345 \
  node scripts/seed-platform.mjs | docker exec -i pub-mail-mysql mysql -upubmail -ppubmail
pubmail
```

O último comando gera o SQL do usuário SUPER_ADMIN e executa dentro do container do MySQL. Não precisa ter o cliente `mysql` instalado. A linha `seed-platform OK` confirma.

Suba a API com recarga automática:

```
yarn start:dev
```

Pronto quando aparecer `Nest application successfully started`. Teste em http://localhost:8000/docs.

4.3 Micro de IA

```
cd ai-micro-services
yarn install
```

Arquivo `ai-micro-services/.env`:

```
PORT=4000
IA_SERVICE_KEY=dev-ia-key
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
OPENROUTER_REFERER=http://localhost:3000
```

```
yarn dev          # com recarga automática (nodemon)
```

4.4 Micro de e-mail

```
cd email-micro-services
yarn install
```

Arquivo `email-micro-services/.env`:

```
PORT=9999
EMAIL_SERVICE_KEY=dev-email-key
REDIS_HOST=127.0.0.1
REDIS_PORT=6379
BASE_URL_API=http://localhost:8000
NEST_WEBHOOK_URL=http://127.0.0.1:8000/webhooks/email
TRACKING_SIGN_SECRET=dev-local-access-secret
```

O `TRACKING_SIGN_SECRET` precisa ser igual ao `JWT_ACCESS_SECRET` do back.

```
yarn start        # este micro não tem modo com recarga; reinicie depois de mudar o código
```

4.5 Front

```
cd front/vite
yarn install
```

Arquivo `front/vite/.env`:

```
VITE_APP_BASE_NAME=/
VITE_API_URL=http://localhost:8000
VITE_APP_VERSION=1.0.0-dev
VITE_STRIPE_ENABLED=false
```

```
yarn start       # abre http://localhost:3000
```

Entre com `admin@pubmail.local` e `admin12345`.

4.6 Depois de puxar código novo

```
# em cada pasta que mudou
yarn install

# no back, se mudou algo em back/prisma
yarn prisma migrate deploy && yarn prisma generate
```

---

5. O que não funciona no ambiente local

Algumas partes dependem de serviços externos ou de um endereço público na internet:

- **IA do webchat** só responde depois de cadastrar uma chave (Groq, Gemini etc.) em Conta & Domínios → Integrações.
- **Envio de e-mail** precisa de uma chave da Resend em Integrações e de um domínio verificado nela.
- **Telegram, PIX e webhooks da Resend** precisam que o Telegram, o gateway de pagamento e a Resend consigam chamar a sua API. `localhost` não é alcançável de fora. Para testar, use um túnel (ngrok, Cloudflare Tunnel) e coloque a URL dele em `PUBLIC_API_URL`.
- **WhatsApp** precisa do gateway OpenWA, que não faz parte deste ambiente. A instalação dele está no `Deploy.md`, seção 22.
- **Domínios de cliente com HTTPS** não são emitidos localmente (`WEBCHAT_DOMAIN_BIND_PROVIDER=NONE`).

O resto do painel funciona normalmente: módulos, cadastros, construtores, leads, CRM e administração.

---

6. Problemas comuns

`yarn -v` mostra 1.22 ou o install falha logo no começo. O Corepack não está ativo. Rode `corepack enable` e abra um terminal novo.

Erro dizendo que o pacote pede Node >= 22 (por exemplo `@prisma/streams-local`). Você está em Node 20 ou mais antigo. Rode `nvm use` na raiz e confira com `node -v`.

`yarn install` falha com "The lockfile would have been modified" ou "Failed with errors". A variável `CI` está definida no seu terminal (mesmo com valor `0`), e isso deixa o Yarn em modo travado. Rode `unset CI` (no PowerShell: `Remove-Item Env:CI`) e instale de novo.

Migration falha com erro 3780 ("incompatible" em foreign key). O banco foi criado com a collation errada. O `docker-compose.yml` atual já cria certo. Recrie o banco com `docker compose down -v` e `docker compose up -d`.

`bind source path does not exist: /etc/timezone` ao subir o Docker. Era um problema da versão antiga do `docker-compose.yml` no Mac e no Windows. Puxe o código atualizado.

`port is already allocated` na 3306, 6379, 8000 ou 3000. Outro programa usa a porta, quase sempre um MySQL instalado na máquina. Pare o outro programa ou troque a porta. Para o MySQL e o Redis, use `MYSQL_PORT` / `REDIS_PORT` no `.env` da raiz e ajuste `DATABASE_PORT` / `DATABASE_URL` no `back/.env`.

`Cannot find module 'generated/prisma/client'` no back. Falta gerar o client: `cd back && yarn prisma generate`.

O back cai no boot falando de Stripe. O `STRIPE_SECRET_KEY` está vazio. Use `sk_test_disabled_placeholder`.

"Too many connections" no MySQL. Falta `DATABASE_POOL_LIMIT=10` no `back/.env`.

O login diz que o usuário ou a senha estão errados. O usuário ainda não foi criado. Rode o comando de seed da seção 4.2. No Jeito B, confira `docker compose logs back | grep dev-start`.

Digitei e-mail e senha, cliquei em Entrar e os campos ficaram vazios. Na primeira abertura o Vite prepara as dependências e recarrega a página uma vez. Espere alguns segundos e entre de novo.

O front abre, mas tudo dá erro de rede ou vai para o servidor de produção. Confira `VITE_API_URL=http://localhost:8000` no `front/vite/.env` e reinicie o `yarn start`, porque o Vite só lê o `.env` ao subir.

Windows. Funciona, mas o caminho mais tranquilo é o WSL2 com Ubuntu: clone o repositório dentro do WSL e siga o guia como se fosse Linux. O Docker Desktop precisa estar com a integração do WSL ligada.

O Docker trava ou o build do front é morto no meio. Falta memória. Em Docker Desktop → Settings → Resources, deixe pelo menos 4 GB.